

Software Test Plan (STP)

Traffic Management System (Simulation)

Project: Traffic Management System (Simulation)

Course: Software Engineering UE24CS341A, PES University

Version: 1.0

Authors: Hariom N Kini (PES2UG24CS672), Deepthi S (PES2UG24CS632), Ramya (PES2UG24CS653), Peddinti Suhas (PES2UG24CS647)

Date: 13-09-2026

Status: Draft

1. Introduction

Purpose: This document defines the test plan for the Traffic Management System (Simulation) v1.0. It outlines objectives, scope, strategy, resources, schedule, and responsibilities for testing the C/C++ console-based application that models vehicle queues at a four-way traffic intersection and controls vehicle movement using traffic signals.

Scope: Testing covers vehicle management, queue management (FIFO per approach), traffic signal management, traffic flow simulation, monitoring/statistics, and optional logging/data management. Real traffic hardware, sensors, cameras, network communication, multiple intersections, GUI rendering, ML-based signal optimization, and database/user-account features are excluded, consistent with the SRS scope.

References: Traffic Management System (Simulation) SRS v1.0, associated Design Specifications, PES University course guidelines (UE24CS341A).

Definitions: SRS (Software Requirements Specification), FR/NFR (Functional/Non-Functional Requirement), FIFO (First In, First Out), Approach (a direction from which vehicles enter the intersection), Phase (a period in which a defined set of approaches has permission to move), Simulation tick (one discrete update of the simulator loop), Throughput (vehicles successfully processed per run duration), STL (C++ Standard Template Library).

2. Test Items

- Vehicle Manager module (vehicle.c)
- Queue Manager module (queue.c) — per-approach FIFO queues
- Traffic Signal Manager module (signal.c)
- Simulation Engine (simulation.c) — tick loop, phase control
- Statistics Manager (statistics.c)
- Logging / File Output Manager (fileio.c) — optional
- Console Interface / Controller (ui.c)

3. Features to be Tested

Features mapped to SRS requirement IDs:

- TMS-F-001 to 005: Vehicle creation, unique IDs, assignment to approach queues, waiting-vehicle display, auto-generation
- TMS-F-006 to 010: Per-approach FIFO queue enqueue/dequeue, empty-queue handling, live count updates
- TMS-F-011 to 015: Signal state display, automatic safe phase cycling, configurable durations, manual signal control, red/yellow blocking
- TMS-F-016 to 021: Tick-based simulation loop, vehicle movement on green, conflict prevention, Start/Pause/Resume/Reset/Stop, invalid-input resilience
- TMS-F-022 to 026: Waiting/processed counts, average waiting time, throughput, end-of-run summary
- TMS-F-027 to 029: Optional configuration save/load and CSV statistics export
- TMS-NF-001 to 007: Performance (100 active vehicles, no noticeable delay), reliability/determinism, usability, maintainability, portability, signal safety, memory safety
- TMS-SR-001 to 005: Input validation, safe empty-queue handling, configuration-file validation, graceful handling of unexpected input, memory de-allocation

4. Features Not to be Tested

- Real traffic lights, sensors, cameras or IoT/field hardware
- Internet/network communication (none exists in the system)
- Multiple intersections or city-wide road networks
- Graphical or windowed rendering (GUI), 2D/3D animation
- Advanced lane-changing, pedestrian, cyclist or public-transport simulation
- Machine-learning signal optimization or real-time adaptive control
- Real-world traffic calibration
- User accounts, cloud storage or a database server

5. Test Approach / Strategy

5.1 Levels

- Unit tests — queue, signal, vehicle and statistics logic tested independently of console I/O via callable functions/test harness
- Integration tests — Queue Manager ↔ Signal Manager ↔ Simulation Engine interactions
- System tests — end-to-end console simulation runs (start → tick → stop)
- Acceptance tests — representative acceptance test suite defined in the SRS (Section 7.3)

5.2 Types

- Functional testing (vehicle, queue, signal, simulation, statistics features)
- Regression testing (after each module change)
- Performance testing (tick processing under up to 100 active vehicles, per TMS-NF-001)
- Usability testing (console menu clarity, per TMS-NF-003)
- Memory-safety testing (Valgrind or equivalent, per TMS-NF-007 / TC-MEM-01)
- Security/robustness testing (input validation, malformed config handling, per TMS-SR-001 to 005)

Entry Criteria: Stable build compiled with the target C/C++ compiler (GCC/G++, Clang or MSVC), test data/scenarios (seeds, generation rates) available, test environment ready.

Exit Criteria: All High-priority functional requirements implemented and their acceptance tests pass; all NFRs checked with no Critical/High defect open; simulation can start, process vehicles, pause, reset and stop via the console menu; final summary shows correct queue and processed counts; no memory leaks or invalid accesses reported by a memory-checking tool.

5.3 Security Validation

- Validate numeric settings (phase durations, generation rates) before applying — reject negative, zero-where-invalid, or non-numeric input (TMS-SR-001)
- Prevent dequeue operations on empty queues via state checks before array/pointer access (TMS-SR-002)
- Validate loaded configuration files, rejecting malformed or incomplete data (TMS-SR-003)
- Confirm unexpected user input does not cause abnormal termination (TMS-SR-004)
- Verify all dynamically allocated memory (e.g., linked-list nodes) is freed on exit and on queue/vehicle removal (TMS-SR-005)

6. Test Environment

Hardware: Standard student laptop/desktop (Windows 10/11 or Linux), keyboard and terminal/console display — no specialized hardware required.

Software: C (C11+) or C++ (C++11+, C++17 recommended) build of the Traffic Management System; standard library only (queue, vector/deque, string, ctime, cstdlib).

Tools: GCC/G++, Clang or MSVC compiler toolchain; Valgrind (or equivalent) for memory-safety checks; a simple unit-test harness for queue/signal/statistics functions; version control and issue tracking as adopted by the team.

Test Data: Predefined PRNG seeds for reproducible vehicle generation, configurable green/yellow duration sets, scripted console input sequences (valid and invalid) for each approach.

7. Test Schedule

Milestones (indicative, within the 1–2 week implementation window defined in the SRS):

- Test case design: within first 2–3 days of implementation start
- Environment/toolchain setup (compiler, Valgrind): Day 2
- Unit test execution (queue, signal, statistics): Days 3–5
- Integration and system test execution: Days 6–9
- Performance, memory-safety and security test execution: Days 9–11
- Acceptance testing / final sign-off: Days 12–14

8. Test Deliverables

- Test Plan (this document)
- Test Cases (unit, integration, system and acceptance — mapped to TC IDs in the SRS RTM)
- Test Scripts / harness code
- Test Data (seeds, configuration files)
- Test Execution Logs

- Defect Reports
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Hariom N Kini (PES2UG24CS672)	Prepare test plan, coordinate execution and reporting
Test Engineer	Ramya (PES2UG24CS653)	Design & execute test cases, log defects
Developer / Test Engineer	Peddinti Suhas (PES2UG24CS647)	Support defect fixes and triage, execute test cases
Project Guide	Deepthi S (PES2UG24CS632)	Approve test results, sign-off readiness

10. Risks and Mitigation

Risk	Mitigation
Short implementation window (1–2 weeks) limits testing time	Prioritize High-priority requirements and acceptance tests first; automate repetitive checks
Memory-safety defects in manual memory management (linked lists)	Run Valgrind (or equivalent) after every significant change, not only at the end
Non-deterministic behaviour from vehicle generation	Use fixed PRNG seeds for reproducible test scenarios (per TMS-NF-002)
Team members work on different modules in parallel	Maintain clear module boundaries (queue.c, signal.c, simulation.c, statistics.c) and integrate early
Portability issues across Windows/Linux compilers	Avoid platform-specific APIs; smoke-test on both environments where available (TC-PORT-01)

11. Assumptions & Dependencies

- A C/C++ compiler and standard library are available on the target machine
- The application starts with a valid default configuration (default green/yellow durations and generation rate)
- Traffic generation uses a simple configurable probability/rate rather than real traffic data

- The four-way intersection uses a predefined safe phase sequence unless manual control is selected
- Testers understand basic traffic-signal concepts

12. Suspension & Resumption Criteria

Suspend testing if:

- The build fails to compile on the supported toolchain
- A blocking defect prevents execution of more than 30% of planned test cases
- Memory-checking tooling (e.g., Valgrind) is unavailable in the environment

Resume testing if:

- Blocking defects are resolved and the build compiles cleanly
- Required tooling is restored

13. Test Case Management & Traceability

The Requirements Traceability Matrix (RTM) in the SRS (Section 8) maps requirements to modules and test cases; status is tracked as N (Not started), P (Partially verified) or A (Accepted).

Req ID	Requirement	Module	Test Case(s)	Status
TMS-F-001-005	Vehicle management	vehicle.c / ui.c	TC-VM-01...05	N
TMS-F-006-010	Queue management	queue.c	TC-Q-01...05	N
TMS-F-011-015	Signal management	signal.c	TC-SIG-01...05	N
TMS-F-016-021	Traffic flow simulation	simulation.c / ui.c	TC-SIM-01...06	N
TMS-F-022-026	Monitoring & statistics	statistics.c / ui.c	TC-MET-01...05	N
TMS-F-027-029	Optional data management	fileio.c	TC-DATA-01...03	N
TMS-NF-001	Performance	simulation.c / ui.c	TC-PERF-01	N
TMS-NF-002	Reliability	All core modules	TC-REL-01	N
TMS-NF-003	Usability	ui.c	TC-UX-01	N

Req ID	Requirement	Module	Test Case(s)	Status
TMS-NF-004	Maintainability	All modules	TC-MAINT-01	N
TMS-NF-005	Portability	Build scripts	TC-PORT-01	N
TMS-NF-006	Safety (no signal conflicts)	signal.c	TC-SAFE-01	N
TMS-NF-007	Memory safety	queue.c / vehicle.c	TC-MEM-01	N
TMS-SR-001–005	Security requirements	ui.c / fileio.c / queue.c	TC-SEC-01...05	N

14. Test Metrics & Reporting

Metrics collected:

- % test cases executed
- % passed / failed
- Defect density and defect aging
- Requirement coverage (per RTM)
- Memory-safety issues found (Valgrind or equivalent)

Reports:

- Daily execution status
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead		
Project Guide		
Team Members		